

JeevanLink — PPT Source Pack

Everything from the codebase and the problem statement that a pitch deck needs, in one place.
Compiled 11 September 2026 from the JeevanLink working repository and SIH26047.pdf.

1. Identity block (title slide)

Field	Value
Team	JeevanSync
Team ID	SIH43
Problem Statement ID	SIH26047 (Problem Statement 4)
Title	Patient Case-Taking Software
Organisation	All India Institute of Ayurveda, Ministry of Ayush
Category / Theme	Software / MedTech · BioTech · HealthTech
Our product name	JeevanLink
In-app tagline	"Your Health. Your Continuity."
In-app section label	"Continuity Centre"
PS's own working name for the solution	"MediKiosk" (tentative, per the PS)
Dataset / reference video supplied by PS	None (both marked NA) — nothing to benchmark against, so the design decisions are ours to defend

2. Problem slide — the numbers (all from the PS document, safe to cite)

- A well-conducted history alone yields the correct diagnosis in **70-80% of cases**, before any examination or investigation. History-taking is the single most important diagnostic activity in clinical medicine.
- Tertiary government hospitals and apex institutions register **4,000-10,000 OPD patients per day**.
- Doctor-to-patient consultation time is routinely **2-5 minutes**.
- **BMJ Open, 2017**, across 67 countries: India's average primary-care consultation is **just over 2 minutes** — among the shortest globally.
- In that window the physician must elicit history, examine, review prior records, diagnose, counsel *and* prescribe. Result: systematic under-elicitation of history, missed comorbidities, repeated questioning across visits, diagnostic error.

- **AYUSH adds a second layer.** Ayurvedic history-taking (Trividha, Ashtavidha, **Dashavidha Pariksha**) requires Prakriti (constitution), Vikriti (current imbalance), Agni (digestive capacity), Koshtha (bowel nature), Ahara-Vihara (diet and lifestyle), Nidana (causative factors) and Samprapti (pathogenesis) — far more extensive than allopathic intake, and effectively impossible inside an OPD slot. Practitioners are forced to abbreviate the very assessment that defines personalised Ayurvedic care.
- **Records are fragmented.** Patients carry paper prescriptions, lab reports, discharge summaries and imaging films from multiple providers — handwritten, multilingual, chronologically disordered — and the physician burns scarce minutes scanning them manually.
- **ABDM exists but the first mile doesn't.** ABHA IDs, the Health Information Exchange and FHIR interoperability are all in place nationally; what's missing is a patient-facing platform that captures structured history and digitises documents *into* that ecosystem **before** the clinical encounter begins.

Why existing solutions fall short (PS's own framing — good "competitive landscape" slide)

Existing approach	Why it fails
Hospital registration systems	Capture only demographics + appointment (name, age, department, token). No clinical history, no document processing.
Mobile health apps / tele-triage chatbots	Require smartphone literacy, stable connectivity and prior enrolment — excluding the elderly, rural, low-literacy and first-visit patients who are the bulk of government OPD load.
Manual nurse-led triage desks	Human-resource-limited; don't scale to 5,000+ patients/day; reintroduce the same transcription bottleneck.
Generic document scanners	Produce images, not structure. No extraction, no chronology, no link to a history or ABHA record.

3. Solution slide — the five-step journey

The PS defines the journey and we implement it end-to-end:

1. **Identify** — patient logs in, selects language, grants consent (audio-guided).
2. **Converse** — AI conducts an adaptive voice + touch history interview; red flags trigger priority triage.
3. **Scan** — patient uploads prior prescriptions, labs and discharge summaries; AI digitises, structures and timelines them.
4. **Summarize & Route** — AI generates the structured history summary and routes it to the clinician queue (and, on the roadmap, to HIS/ABHA via FHIR).
5. **Consult** — physician reviews a complete structured history in seconds, edits/confirms, and spends the visit on examination, reasoning and counselling.

One-line pitch: *JeevanLink does the history-taking the OPD has no time for — in the patient's own language, by voice or touch, before they ever reach the consultation room.*

4. Architecture & tech stack slide

Frontend / framework - Next.js **16.3.3** (App Router, server components + client components) - React **19.2.8**, TypeScript **5**, Tailwind CSS **4** - `lucide-react` for iconography

Backend / data - Next.js Route Handlers (`runtime = "nodejs"` , `maxDuration` 60–120s on AI routes) - Prisma ORM **6.19.3**, SQLite dev database (`prisma/dev.db`), **4 migrations** - Auth: `bcryptjs` password hashing + HTTP-only cookie session (`jeevanlink_session`), server-side `requireRole()` guard on every protected route - File storage: `private-uploads/medical/` with UUID filenames, served only through an authenticated route handler — uploads are never public static assets

AI layer — Google Gemini via @google/genai 2.19 | Purpose | Model | Notes | |---|---|---| | Medical document extraction (OCR + structuring) | `gemini-3.6-flash` | multimodal, strict `responseSchema` JSON | | Speech-to-text for intake answers | `gemini-3.6-flash` | audio in, transcript out, language-pinned | | Text-to-speech for audio prompts | `gemini-3.1-flash-tts-preview` | "Kore" voice, PCM→WAV assembled server-side | | Clinician SOAP scribe | `gemini-3.6-flash` | structures dictated notes | | Personalized-health explanation layer | `gemini-3.6-flash` | explains deterministic analytics, never diagnoses | | Patient voice assistant | `gemini-3.6-flash` | answer / navigate / action-draft, all server-validated |

Quality tooling - Vitest **3.2.7** + `vite-tsconfig-paths` ; `npm test` / `npm run test:watch` - ESLint 9 + `eslint-config-next`

5. Feature inventory, mapped to the PS's four modules

Module A — Conversational multimodal history engine BUILT

- `src/app/clinical-intake/page.tsx` (**73.8 KB**) — the intake flow itself
- Dual-mode input: every question answerable by **speaking or tapping**
- `api/clinical-intake/transcribe` — records an answer, transcribes it in the language it was spoken, explicitly instructed *not* to translate and to return an empty string rather than guess at unclear audio
- `api/clinical-intake/speak` — reads prompts aloud in the patient's language across 12 Indian speech locales (`hi-IN` , `ta-IN` , `bn-IN` , ...)
- **AYUSH mode** — extended Dashavidha Pariksha interview (Prakriti, Vikriti and the rest) for Ayurvedic OPDs, carried through to the stored record as a dedicated `ayush` block
- **Red-flag detection** — `src/app/lib/redFlags.ts` , 5 emergency categories, **~200 terms across all 12 languages**: 1. Chest pain or chest pressure 2. Severe breathing difficulty 3. Loss of consciousness / fainting 4. Possible acute neurological symptom (facial droop, slurred speech, sudden weakness/numbness, seizure) 5. Severe bleeding (including haematemesis, haemoptysis)

Module B — Medical document digitisation BUILT

- `api/upload-document` — PDF/JPG/JPEG/PNG/WEBP, 15 MB cap, UUID-named private storage
- `api/analyze-document` (**23.4 KB**) — Gemini multimodal extraction with an enforced JSON schema returning:
 - `documentType` (prescription / lab / X-ray / MRI / CT / ultrasound / discharge summary / other)
 - `patient` (name, age, sex, patient ID), `documentDate`
 - `medications[]` — name, dosage, frequency, duration, instructions
 - `labResults[]` — test name, value, unit, **reference range, normal/high/low status**
 - `radiology` — examination, body region, clinical history, technique, findings[], impression[]
 - `warnings[]`
- Handles **printed and handwritten**, multilingual
- `health-timeline` page (**44 KB**) — chronological view across records

Module C — Structured history summary BUILT

- Standard clinical format stored per intake: chief complaint → HPI → past medical → past surgical → medications → allergies → family → personal → review of systems → prior investigations → AYUSH block → red flags → safety note
- **Bilingual output implemented:** the intake route runs a best-effort English translation pass so the physician console reads English regardless of the language the patient answered in — and is explicitly written to *never block submission* if that pass fails (quota/network), saving the original-language answers regardless
- Clinician surfaces: `clinician/page.tsx` (**115 KB**), `ClinicalSnapshotPanel`, `ClinicalHistoryPanel`, `VitalTrendPanel`
- `api/scribe/generate` — dictated clinician notes → structured **SOAP** draft, with "do not invent findings, diagnoses, medications or values" hard-coded into the prompt

Module D — Consent, privacy, ABDM CONSENT BUILT ABDM ROADMAP

- **Append-only `ConsentEvent` log** — a grant or revoke is a *new row*, never an overwrite, so every change stays independently auditable. Current status for a category = most recent event for that (patient, category) pair.
- 5 consent categories: `CLINICAL_HISTORY`, `DOCUMENT_PROCESSING`, `CLINICIAN_SHARING`, `RESEARCH_DATA_SHARING`, `REMINDERS_NOTIFICATIONS`
- Intake enforces consent server-side: no clinical-history consent or no clinician-sharing consent ⇒ **400, submission refused**
- Consent checkboxes collected at intake are mirrored into the same auditable log, so the consent centre reflects real activity
- ABHA identity + FHIR push to HIS/PHR: **roadmap** (nav entry "FHIR / ABDM" exists; auth is currently email + password)

Beyond the PS — the continuity platform we built around it

Area	What's there
Vitals	8 vital types, 5 sources incl. Google Health Connect / Samsung Health / wearables; <code>vitalAnalytics.ts</code> (22 KB) does validation, scoring, trend and health-summary computation; <code>vitals/ai-report</code> generates a narrative report
Medications	Medication records, reminder schedules, and taken/skipped/snoozed adherence logs
Prescriptions	Dedicated prescriptions view (48 KB)
Hospitals & Labs	Admissions, encounters, tests, procedures, bills, bill items, insurance claims, lab orders, lab reports (91 KB page + 34 KB API)
Personalized Health	Deterministic analytics first, AI explanation layer second, with explicit data-gap surfacing
Voice assistant	App-wide launcher; three response types — answer, navigate, action-draft (log a vital)
Helpdesk	Ticketing with priorities, categories, assignment, threaded messages, callback requests
Admin	Overview of users by role, record status breakdown, ticket pipeline, consent event counts

6. "Why we're safe" slide — the strongest differentiator for judges

Every one of these is real, in-code, and quotable:

- **No AI output is ever final.** Every intake and every extracted document lands as `status: PENDING` and requires an explicit clinician **VERIFY** or **REJECT**. `VerificationAudit` logs every action with actor and note.
- **Rejection is a closed loop.** A `REJECTED` record *requires* a correction note; the patient sees it, corrects, and resubmits — the record round-trips instead of dying.
- **Structured-output contracts.** Every Gemini call uses `responseMimeType: application/json` with a typed `responseSchema`; free-form model prose never reaches the database.
- **Anti-hallucination prompt contracts.** "Do NOT diagnose." "Do NOT invent missing information." "Preserve uncertainty when text is unclear." "If a field is unavailable, return an empty string." "This output is a draft for clinician verification."
- **A stored safety note on every record:** *"This is a patient-reported clinical intake draft. It is not a diagnosis or treatment recommendation and requires clinician verification."*
- **The model can't act on its own.** The voice assistant's navigation targets are validated against a server-side whitelist and action drafts against a fixed vital-type list — the code comments it plainly: *never forward a navigation target or an action draft the model invented*.
- **Emergency detection deliberately doesn't depend on AI.** Red-flag matching is deterministic keyword matching, so the emergency path can't fail because of an API outage, a quota exhaustion or added latency. (Documented trade-off: it will miss paraphrases; a Gemini second pass is a considered, deferred upgrade.)

- **Graceful degradation everywhere.** Retry with exponential backoff, and typed error codes — `AI_QUOTA_EXCEEDED` , `AI_SERVICE_UNAVAILABLE` , `EMPTY_AI_RESPONSE` , `INVALID_AI_RESPONSE` — each with a patient-facing fallback ("please type your answer instead").
 - **Role-gated everything.** `requireRole("PATIENT" | "CLINICIAN" | "HELPDESK" | "ADMIN")` on every protected route; uploaded documents are served only through an authenticated handler.
-

7. Inclusion slide — language and accessibility

12 languages, each with native script and an Indian speech locale:

English · हिन्दी · বাংলা · தமிழ் · తెలుగు · मराठी · ગુજરાતી · ಕನ್ನಡ · മലയാളം · ਪੰਜਾਬੀ · ଓଡ଼ିଆ · অসমীয়া

- `i18n.ts` is **4,221 lines / 437 KB** — the full UI translated, not a token language switcher
 - **Red flags are multilingual too.** A Hindi or Tamil speaker's emergency report is caught by terms in their own script — the code comment explains why this matters: English-only matching "*would silently miss every non-English emergency report.*"
 - **Accessibility toggles** (`AccessibilityProvider`): **high contrast, large text, audio-guided mode** — persisted per patient in local storage
 - **Audio-guided consent** for low-literacy patients, and every question answerable by voice or touch, so zero digital literacy is required
-

8. Data model slide

21 models, 17 enums. Headline models:

`User` (4 roles) · `MedicalRecord` (PENDING/VERIFIED/REJECTED) · `Medication` · `MedicationReminder` · `MedicationLog` · `VerificationAudit` · `VitalMeasurement` · `Hospital` · `HospitalAdmission` · `HospitalEncounter` · `HospitalTest` · `HospitalProcedure` · `HospitalBill` · `HospitalBillItem` · `InsuranceClaim` · `Lab` · `LabOrder` · `LabReport` · `ConsentEvent` · `Ticket` · `TicketMessage`

Worth calling out on the slide: the **append-only ConsentEvent** design and the **VerificationAudit** trail — those two are what make the system defensible under DPDP scrutiny.

9. "What we built" metrics slide

Metric	Value
Application pages	18
API / route handlers	30
React components	9 shared + page-level
Library modules	7
Database models / enums	21 / 17
Prisma migrations	4
Languages supported	12
Test suites	3 (Vitest: intake, document analysis, consent)
Total source	≈1.8 MB across ~72 files
Largest surfaces	clinician console 115 KB · hospitals & labs 91 KB · vitals 76 KB · clinical intake 74 KB
Build window	active development since 28 Aug 2026 (first migration) — ~2 weeks

(Sizes are file bytes, which is exact; line counts quoted anywhere should be treated as approximate except `i18n.ts` = 4,221 lines.)

10. Compliance mapping slide

Requirement	Status
DPDP Act 2023 — granular, revocable consent	BUILT 5 categories, revocable, every change logged
DPDP — auditability	BUILT append-only consent log + verification audit trail
DPDP — purpose limitation	BUILT consent is per-purpose, enforced server-side at submission
Consent explained to low-literacy patients	BUILT audio-guided consent
ABDM — ABHA identity	ROADMAP roadmap (currently email + password)
ABDM — FHIR push to HIS / PHR	ROADMAP roadmap (nav entry present, integration pending)
Secure processing of health data	BUILT role-gated APIs, private authenticated document storage, hashed credentials

11. Honest status slide (recommended — judges reward this)

Demo-ready today: multilingual voice + touch intake with AYUSH mode · multilingual red-flag triage · document upload → OCR → structured extraction · clinician verification queue with correction loop · consent centre with audit log · patient dashboard, vitals, medications, prescriptions, timeline · AI scribe · voice assistant · helpdesk · admin overview.

Next up: ABHA-based identification · real FHIR/ABDM push to HIS and Personal Health Record · model-assisted second-pass red-flag detection for paraphrases · Postgres + hosted deployment.

Explicitly deferred (and why): sign-language avatar — the PS itself designates it a stretch goal; Bhashini/AI4Bharat ASR — Gemini's audio understanding covers the same need today, and we'd want real noisy-OPD recordings before committing to a swap.

12. Suggested slide order (13 slides)

1. Title — team, PS ID, product name, tagline
 2. The 2-minute consultation (problem stats)
 3. The AYUSH complication (Dashavidha Pariksha vs. OPD time)
 4. Fragmented records + the ABDM first-mile gap
 5. Why existing solutions fall short (the 4-row table)
 6. Solution overview — the five-step journey (**use the flowchart artifact**)
 7. Module A — conversational multimodal intake (voice/touch, 12 languages, AYUSH mode)
 8. Module B — document digitisation (extraction schema + timeline)
 9. Module C — structured summary + clinician verification loop
 10. Module D — consent, privacy, ABDM roadmap
 11. Architecture & stack
 12. Why it's safe — AI guardrails (the differentiator slide)
 13. Impact, status and roadmap
-

13. Judge Q&A prep

"Is the AI diagnosing patients?" No. Every output is a draft in **PENDING** until a clinician verifies it, every prompt forbids diagnosis, and a safety note is stored on the record itself. The clinician can reject with a correction note, which sends it back to the patient.

"What if the AI hallucinates a medication?" Three defences: a typed response schema so only declared fields come back, prompts that require empty values over guesses, and a clinician verification gate before anything is trusted.

"What happens when the API is down or out of quota?" Typed error codes and fallbacks — voice fails over to typing, document analysis returns a clear retry message. Critically, **red-flag emergency detection never calls the AI at all**, so triage can't be taken down by an outage.

"Does this work for a patient who can't read?" Audio-guided consent and prompts, every question answerable by speaking, high-contrast and large-text modes, and 12 languages in native script.

"How is this different from a chatbot?" A chatbot needs a smartphone, connectivity and prior enrolment. This is a kiosk-first, zero-training, walk-up flow designed for the first-visit patient — plus it digitises the paper they're carrying, which no chatbot does.

"Is ABDM integration real?" Not yet — that's stated plainly on the roadmap slide. The consent architecture underneath it (granular, revocable, append-only, purpose-scoped) is already built to what ABDM's consent framework and DPDP require, which is the harder half.

Provenance note

API route handlers, `redFlags.ts`, `AccessibilityProvider.tsx`, `schema.prisma`, `auth.ts` and `package.json` were read in full. The largest page components (clinician 115 KB, hospitals-labs 91 KB, vitals 76 KB, clinical-intake 74 KB) were keyword-scanned rather than read end-to-end, so feature descriptions for those surfaces are accurate at the level of what they contain, not exhaustive about every interaction.